



Посібник, який має навчити: ТЗНК, English, IT

Версія v16: не суха шпаргалка, а пояснення під реальні тестові пастки. Фокус: розпізнати тип питання, вибрати правильний алгоритм, не переплутати схожі поняття.

Як читати: спочатку піддисципліна, потім алгоритм рішення, потім пастки. Якщо питання здається страшним, знайди в ньому маркер: АБО/І, порядок, база відсотка, граматична форма, роль IT-компонента.

КАРТА



Які блоки закриває посібник

Блок 01

 ТЗНК: комбінаторика, логіка, відсотки, висновки

Блок 02

гв English: grammar, use of English, reading evidence

Блок 03

 IT: programming, OOP, algorithms, databases, networks, OS, security, ML

Головна ідея: тест не питає "чи знайоме слово". Він питає, чи можна відрізнити сусідні поняття. Тому в кожному блоці є "де ловлять".

БЛОК 01



ТЗНК: комбінаторика, логіка, відсотки, ВИСНОВКИ

Мета: Навчитися не вгадувати формулу, а розпізнавати тип задачі за словами: або, і, порядок, без повторів, хоча б один, істинне/хибне.

АБО → сума

І / послідовність → добуток

порядок важливий

порядок не важливий

обмеження

must be true

Науково: ТЗНК перевіряє формалізацію умови. Треба перетворити текст на структуру: вибір з альтернатив, послідовність дій, множину без порядку, логічне твердження або відсоткову зміну від бази.

Приклад у житті: Уяви меню. Якщо можна взяти каву або чай, варіанти додаються. Якщо треба взяти напій і десерт, варіанти множаться. Якщо обираємо комісію з людей, порядок не важливий. Якщо призначаємо президента і секретаря, порядок важливий.

Піддисципліни всередині блоку

Комбінаторика: правило суми

Правило суми працює, коли треба обрати один варіант з кількох неперетинних груп. Маркери: або, один із, будь-який з цих об'єктів.

а має 6 способів, b має 4 способи.
Обрати один: а або b.
Це не пара (a,b), а одна доріжка.
 $6 + 4 = 10$

Комбінаторика: правило добутку

Правило добутку працює, коли треба виконати кілька кроків разом: спочатку А, потім В.
Маркери: і, пара, послідовність, код, маршрут.

3 сорочки і 4 пари штанів.
Образ = сорочка + штани.
 $3 * 4 = 12$

Перестановки та розміщення

Якщо порядок важливий, А-В і В-А різні. Для k місць з n об'єктів без повторів: $n*(n-1)*...$ стільки множників, скільки місць.

Президент і секретар з 5 людей:
5 варіантів на президента
4 варіанти на секретаря
 $5 * 4 = 20$

Комбінації

Якщо порядок не важливий, А-В і В-А однакова група. Тоді ділимо на кількість перестановок усередині вибраної групи.

Обрати 3 людей з 8:
 $C(8,3) = 8*7*6 / (3*2*1) = 56$

Обмеження та заборонені випадки

Спершу рахуємо всі можливі варіанти, потім віднімаємо заборонені. Це зручно, коли заборону легше порахувати, ніж дозволені варіанти.

Комісія з 3 людей з 8, А і В не разом.
Усі: $C(8,3) = 56$
Заборонені: А і В + 1 з решти 6 = 6
Дозволені: $56 - 6 = 50$

Логіка: якщо Р, то Q

Імплікація $P \rightarrow Q$ хибна тільки в одному випадку: Р істинне, а Q хибне. Якщо умова не виконалась, обіцянка формально не порушена.

Якщо прибереш кімнату (P),
отримаєш цукерку (Q).
Порушення тільки так:
прибрав, але цукерку не отримав.

Відсотки

Відсоток завжди рахується від поточної бази. +20%, а потім -20% не повертає до початкового числа.

$100 + 20\% = 120$
 $120 - 20\% = 96$
Не 100, бо друга база вже 120.

Текстові висновки

Must be true означає: відповідь обов'язково впливає з умови. Не можна додавати здоровий глузд, симпатичну версію або припущення.

Якщо $A > B$ і $B > C$,
то $A > C$ обов'язково.
А от 'А найкращий' не впливає.

Як розв'язувати на тесті

П'ять питань перед формулою

Перед обчисленням постав ці питання. Вони майже завжди показують, що робити: додавати, множити, ділити на перестановки або віднімати заборонене.

1. Обираємо одне чи кілька?
2. Є слово АБО чи І?
3. Порядок важливий?
4. Повтори дозволені?
5. Є заборона / 'хоча б один'?

Як вирішити приклад а або b

Умова каже: обрати один. Це не означає взяти а і b разом. Є 6 доріжок через а і 4 доріжки через b.

a: a1 a2 a3 a4 a5 a6
b: b1 b2 b3 b4
Разом доріжок: $6+4=10$

Хоча б один

Часто легше рахувати через доповнення: усі варіанти мінус жодного потрібного.

У паролі має бути хоча б одна цифра.
Дозволені = усі паролі – паролі без цифр.

Логічна імплікація

Не думай морально, думай формально. Обіцянку порушено тільки коли умова виконана, а результат не настав.

P true, Q true → правило true
P true, Q false → правило false
P false, Q true → правило true
P false, Q false → правило true

Типові пастки: де ловлять

- **Множать там, де треба додати.**

Якщо в умові 'а або b', це альтернативи. Ми не створюємо пару a-b, тому $6*4$ буде пасткою.

Обрати будь-який з об'єктів а або b → $6+4$.

- **Додають там, де треба множити.**

Якщо один результат складається з кількох частин, кожна перша опція поєднується з кожною другою.

Логін і пароль, сорочка і штани, маршрут з двох ділянок.

- **Не ділять комбінації на повтори порядку.**

Комісія {A,B,C} не змінюється від перестановки B,C,A. Якщо порядок не важливий, перестановки всередині групи треба прибрати.

$C(8,3)=8*7*6/(3*2*1)$, не $8*7*6$.

- **Must be true плутають з 'може бути правдою'.**

У тесті треба брати лише те, що гарантовано впливає з умови. Можливі життєві пояснення не рахуються.

Якщо сказано 'деякі', не можна робити висновок 'усі'.

- **Відсотки рахують від старої бази.**

Після кожної зміни база стає новою. Тому +20% і -20% не компенсують одне одного.

100 → 120 → 96.

Міні-дрил

Дрил 1: АБО чи І

Підкреслюй у задачі слово, яке вирішує дію.

один із / або / будь-який → додати
пара / код / послідовність / і →
помножити

Дрил 2: порядок

Питай: якщо поміняти місцями, це новий результат?

президент-секретар: так → порядок
важливий
комісія з 3 людей: ні → комбінація

Дрил 3: обмеження

Якщо є 'не разом', 'хоча б один', 'без', часто вигідно рахувати через заборонене.

дозволені = усі – заборонені

БЛОК 02

GB English: grammar, use of English, reading evidence

Мета: Зібрати граматику не як список часів, а як систему маркерів: форма дієслова, часова рамка, прийменник, linker, доказ у тексті.

tenses

passive

conditionals

modals

collocations

reading

Науково: Use of English перевіряє, чи підходить форма слова граматично й лексично. Reading перевіряє не пам'ять, а доказ: правильна відповідь має опору в тексті.

Приклад у житті: Речення як пазл. Якщо одна деталь не підходить за формою, весь пазл кривий: час, прийменник, артикль, linker і сенс мають стати разом.

Піддисципліни всередині блоку

Tenses: часова рамка

Час показує, коли дія відбувається і чи важливий результат. Не перекладай дослівно, шукай маркери: yesterday, already, since, while, by the time.

Past Simple: I studied yesterday.
Present Perfect: I have studied this topic already.
Past Continuous: I was studying when she called.

Passive voice

Passive потрібен, коли важливий об'єкт дії, а не виконавець.
Формула: be + V3.

The committee reviewed the report.
The report was reviewed by the committee.

Conditionals

Conditionals показують тип умови: факт, реальна майбутня умова, нереальна теперішня, нереальна минула.

- 0: If water boils, it evaporates.
- 1: If I study, I will pass.
- 2: If I studied more, I would pass.
- 3: If I had studied, I would have passed.

Modals

Modals показують силу твердження: must - необхідність або сильний висновок, may/might - можливість, should - порада, can/could - здатність або можливість.

You must wear a helmet. = obligation
She might be late. = possibility
He must be tired. = strong deduction

Collocations and prepositions

У тестах часто питають сталі поєднання. Їх не завжди можна вивести логікою з української.

depend on
responsible for
interested in
similar to
different from
pay attention to
take part in

Reading evidence

Правильна відповідь не має бути просто 'розумною'. Вона має бути підтверджена фразою в тексті або дуже близьким перефразуванням.

Text: The number declined slightly.
Correct paraphrase: It decreased a little.
Wrong: It collapsed dramatically.

Як розв'язувати на тестах

Use of English: порядок дій

Спершу визнач роль пропуску: дієслово, іменник, прикметник, прийменник, linker. Потім перевір граматику зліва і справа.

The report ___ by the committee.
by the committee → passive
was reviewed

Reading: доказ перед відповіддю

Перед вибором знайди в тексті речення-доказ. Якщо доказу немає, відповідь може бути пасткою.

Question: What can be inferred?
Good answer = обережний висновок з тексту
Bad answer = додана фантазія

Linkers

Linker змінює логіку речення:
contrast, cause, result, addition. Не
дивись тільки на переклад, дивись
на відношення між частинами.

although + clause
however, + sentence
despite + noun/gerund
because + reason
therefore + result

Типові пастки: де ловлять

- **Second і Third Conditional плутають.**

Second говорить про нереальну теперішню/майбутню ситуацію. Third говорить про минуле, яке вже не змінити.

If I had known, I would have helped. Це third.

- **Present Perfect плутають з Past Simple.**

Past Simple має завершений час у минулому. Present Perfect часто має результат зараз або період, що триває до тепер.

I lost my key yesterday. / I have lost my key, so I can't enter.

- **Дослівний переклад прийменників.**

English collocations часто не збігаються з українською логікою.
depend on, не depend from.

- **Reading inference роблять занадто широким.**

Inference має бути обережним. Якщо в тексті 'some', не можна відповідати 'all'. Якщо 'slightly', не можна 'dramatically'.

probably / suggests / may indicate - обережніші за proves.

Міні-дрил

Дрил 1: conditionals

Дивись на форму після if.

present → 0/1

past → 2

had + V3 → 3

Дрил 2: reading

Для кожної відповіді пиши поруч слово-доказ з тексту.

Answer A → line 4: 'however'

Answer B → no evidence → reject

Дрил 3: collocations

Вчи не окреме слово, а пару.

responsible FOR

interested IN

similar TO

capable OF

БЛОК 03

IT: programming, OOP, algorithms, databases, networks, OS, security, ML

Мета: Покрити IT як карту піддисциплін: що означає поняття, де воно живе в системі, чим відрізняється від схожого, на чому ловлять у тестах.

programming

OOP

algorithms

SQL

networks

OS

security

ML

Науково: IT-тест перевіряє базову комп'ютерну грамотність на рівні понять і ролей: що робить компонент, на якому рівні працює, які дані приймає, який результат дає.

Приклад у житті: Думай про IT як про місто. Програми - це сервіси, БД - архів, мережа - дороги й адреси, ОС - диспетчер ресурсів, security - замки й правила доступу, ML - модель, яка вчиться на прикладах.

Піддисципліни всередині блоку

Procedural programming

Процедурне програмування описує послідовність кроків: змінні, умови, цикли, функції. Питання часто питають, що буде після виконання коду.

```
let x = 2;  
x = x + 3;  
if (x > 4) x = x * 2;  
// x = 10
```

Compiler, interpreter, linker

Compiler перекладає код у машинний/об'єктний код.
Interpreter виконує код поступово.
Linker збирає кілька об'єктних модулів в один виконуваний модуль.

```
source code → compiler → object  
files  
object files + libraries → linker  
→ executable
```

OOP

Class - шаблон. Object - конкретний екземпляр. Encapsulation ховає внутрішній стан. Inheritance моделює is-a. Polymorphism дозволяє однаковий виклик з різною поведінкою.

```
class Animal { speak(){} }  
class Dog extends Animal { speak()  
{ return 'woof'; } }  
class Cat extends Animal { speak()  
{ return 'meow'; } }
```

Algorithms and Big-O

Big-O описує, як зростає кількість операцій при збільшенні вхідних даних. Не рахує точний час, а форму росту.

```
O(1) direct access  
O(log n) binary search  
O(n) one loop  
O(n^2) nested loops
```

Data structures

Array дає швидкий доступ за індексом. Stack працює LIFO. Queue працює FIFO. Hash table шукає за ключем. Tree має ієрархію.

```
stack: last in, first out  
queue: first in, first out  
hash table: key → value
```

Databases and SQL

Relational DB зберігає дані в таблицях. Primary key унікально визначає рядок. Foreign key посиляється на іншу таблицю. GRANT дає права доступу.

```
SELECT department, COUNT(*)  
FROM employees  
WHERE active = true  
GROUP BY department  
HAVING COUNT(*) > 5;
```

Networks

Switch пересилає кадри за MAC-адресами в LAN. Router маршрутизує пакети за IP. DNS перетворює домен на IP. DHCP видає мережеві налаштування.

```
switch → MAC / layer 2  
router → IP / layer 3  
TCP → reliable  
UDP → fast, no delivery guarantee
```

Operating systems

OS керує процесами, потоками, пам'яттю, файлами, правами доступу й пристроями введення/виведення.

```
process = running program  
thread = execution path inside  
process  
RAM = volatile memory  
file system = organized storage
```

Cybersecurity and cryptography

Hash односторонній відбиток.
Encryption зворотне шифрування з ключем. Authentication перевіряє хто ти. Authorization перевіряє що тобі дозволено.

Купуна = hash function
AES / Rijndael = block cipher
TLS protects HTTPS
least privilege = мінімум прав

Data Science / ML

Classification прогнозує клас.
Regression прогнозує число.
Clustering групує без міток. PCA зменшує розмірність. SVM шукає межу з максимальним margin.

spam / not spam → classification
price tomorrow → regression
customer groups → clustering

Як розв'язувати на тесті

Як відповідати на IT-визначення

Знайди ключову дію в питанні:
збирає, виконує, маршрутизує,
шифрує, хешує, нормалізує,
класифікує.

збирає object modules → linker
виконує code line by line → interpreter
translates source code → compiler

Як не плутати мережі

Запам'ятай ролі, не назви. Switch всередині LAN, router між мережами, DNS для імен, DHCP для налаштувань.

domain → DNS → IP
new laptop joins Wi-Fi → DHCP gives IP
packet to another network → router

Як не плутати security

Hash не розшифровують. Encryption розшифровують. Signature доводить автора/цілісність. TLS захищає канал.

password storage → hash
private message → encryption
software release proof → digital signature

Як читати код у тесті

Створи маленьку таблицю значень змінних після кожного рядка. Не тримай все в голові.

i		x
0		1
1		2
2		4
3		8

Типові пастки: де ловлять

- **Class і object плутають.**

Class - шаблон. Object - конкретний створений екземпляр.

Dog як порода/тип - class; конкретний пес Бім - object.

- **Inheritance плутають з composition.**

Inheritance = is-a. Composition = has-a / part-of.

Car has Engine, але Car is not Engine.

- **WHERE і HAVING міняють місцями.**

WHERE фільтрує рядки до групування. HAVING фільтрує групи після GROUP BY.

WHERE active=true; HAVING COUNT(*)>5.

- **Switch і router плутають.**

Switch працює з MAC у локальній мережі. Router працює з IP між мережами.

Комутатор у LAN пересилає кадри за MAC.

- **Hash і encryption плутають.**

Hash не має нормального 'розшифрування'. Encryption має зворотне перетворення з ключем.

Купуна - hash; AES/Rijndael - cipher.

- **Classification і regression плутають.**

Classification дає категорію. Regression дає число.

хворий/здоровий → classification; температура завтра → regression.

Міні-дрил

Дрил 1: ролі систем

Для кожного терміна пиши одну дію.

compiler → translates
linker → combines
router → routes
DNS → resolves
GRANT → grants permissions

Дрил 2: SQL-порядок

Тримай порядок виконання як ланцюжок.

FROM → WHERE → GROUP BY → HAVING →
SELECT → ORDER BY

Дрил 3: ML-питання

Питай, який тип відповіді очікується.

```
class label → classification  
numeric value → regression  
hidden groups → clustering  
reduce variables → PCA
```

ФІНАЛЬНА КАРТА



Чекліст перед тестом

ТЗНК

Перед формулою відповісти: АБО чи І? порядок важливий? повтори дозволені? є заборона? це must be true чи may be true?

OR -> add
AND/sequence -> multiply
order matters -> arrangements
order not matters -> combinations
forbidden -> total - forbidden

English

Перед відповіддю знайти граматичну роль і доказ у тексті. Не перекладати прийменники дослівно.

if + had V3 -> third conditional
be + V3 -> passive
although + clause
despite + noun/gerund

IT

Для кожного терміна назвати роль. Якщо роль інша, відповідь інша, навіть якщо слово звучить знайомо.

switch -> MAC
router -> IP
DNS -> name to IP
GRANT -> permissions
hash -> one-way
classification -> class

Як тренуватися

Після кожної помилки записати не тільки правильну відповідь, а тип пастки.

1. confused terms
2. missed keyword
3. wrong formula
4. added assumption
5. guessed grammar



Карта підготовки: що саме треба вміти на тесті

Мета: не просто впізнавати правильну відповідь, а розуміти, за якою ознакою вона правильна. На реальному тесті часто перевіряють не термін, а межу між двома схожими термінами.



ТЗНК

Логіка висловлювань, комбінаторика, відсотки, таблиці, графіки, висновки з тексту. Головне питання: що саме дозволено умовою, а що ми додумали самі?

GB English

Grammar in context, Use of English, reading for evidence, linkers, collocations. Головне питання: яка граматична роль слова в реченні і де доказ у тексті?



IT

Програмування, ООП, алгоритми, архітектура, ОС, мережі, БД, безпека, ML/Data Science. Головне питання: яку роль виконує об'єкт, команда або технологія?

Науково: екзамен будується на класифікації. Якщо поняття мають схожі слова, тест перевіряє ознаку розрізнення: порядок важливий чи ні, дія одна чи послідовна, права доступу чи запит даних, switch чи router, inheritance чи composition.

Приклад у житті: якщо в меню написано "кава або чай", це правило суми. Якщо написано "напій і десерт", це правило добутку. Якщо "обери 3 людей у комісію", порядок не важливий. Якщо "президент, секретар, бухгалтер", порядок важливий, бо ролі різні.



ТЗНК: комбінаторика без паніки

Фокус: навчитися бачити дію в умові: "або", "і", "порядок важливий", "порядок не важливий", "заборонено", "принаймні", "рівно".

Піддисципліни всередині блоку

Правило суми

Використовуємо, коли треба обрати один варіант з кількох неперетинних груп: А або В.
Варіанти не виконуються разом.

а можна вибрати 6 способами, б – 4 способами.
Обрати один: а або б.
Разом: $6 + 4 = 10$

Правило добутку

Використовуємо, коли вибір складається з кількох кроків: спочатку А, потім В. Кожен варіант А може поєднатися з кожним варіантом В.

3 напої і 4 десерти.
Обрати напій і десерт.
Разом: $3 * 4 = 12$

Перестановки

Порядок важливий, і використовуються всі об'єкти. Якщо 4 людини стають у чергу, А-Б-В-Г і Б-А-В-Г - різні варіанти.

4 людини в чергу:
 $4! = 4 * 3 * 2 * 1 = 24$

Розміщення

Порядок важливий, але використовуються не всі об'єкти. Наприклад, з 8 людей треба обрати президента і секретаря.

$A(8,2) = 8 * 7 = 56$
бо перша роль має 8 варіантів, друга – 7 після першого вибору

Комбінації

Порядок не важливий. Комісія {А, Б, В} - це та сама комісія, що {В, Б, А}. Тому ділимо на кількість перестановок усередині групи.

$$C(8,3) = 8 * 7 * 6 / (3 * 2 * 1) = 56$$

Заборонені варіанти

Спочатку рахуємо всі варіанти, потім віднімаємо ті, які порушують умову. Це часто найпростіший шлях.

Комісія з 3 людей із 8, А і Б не можуть бути разом.
Усі: $C(8,3)=56$
Погані: А і Б уже взяті, третій з решти 6
Правильні: $56 - 6 = 50$

Як розв'язувати задачу "а - 6 способів, b - 4 способи, обрати один"

Крок 1: знайти ключове слово

Умова каже: обрати один: а або b. Слово 'або' означає, що ми не будемо пару (a,b), а відкриваємо дві окремі доріжки.

доріжка А: 6 варіантів
доріжка В: 4 варіанти

Крок 2: перевірити, чи дії одночасні

Якщо треба було б обрати а і b, тоді було б $6*4$. Але тут треба обрати будь-який один об'єкт з двох типів.

не пара: (a,b)
а один вибір: або а, або b

Крок 3: скласти доріжки

Коли доріжки альтернативні, додаємо. Коли кроки послідовні, множимо.

$$6 + 4 = 10$$

Пастка

Тест часто дає варіант 24, бо мозок бачить два числа і хоче множити. Але множення - тільки коли обидва вибори потрібні одночасно.

обрати а або b $\rightarrow +$
обрати а і b $\rightarrow *$

- Слова 'будь-який з цих об'єктів' часто означають суму.
Бо треба один об'єкт з об'єднання груп, а не набір з двох об'єктів.
 m способів для а, n способів для b $\rightarrow m+n$
- Слова 'пара', 'послідовність', 'код', 'маршрут' часто означають добуток.

Бо результат складається з кількох позицій, і кожна позиція має свої варіанти.

2 літери і 3 цифри -> $26 \cdot 26 \cdot 10 \cdot 10 \cdot 10$

- **'Не разом', 'без', 'крім' часто зручно рахувати через віднімання.**

Усі варіанти мінус заборонені майже завжди простіше, ніж будувати правильні з нуля.

- **'Принаймні один' часто зручно рахувати через доповнення.**

Принаймні один = усі варіанти мінус жодного.

хоча б одна дівчина = усі команди - команди без дівчат



ТЗНК: імплікація, висновки, таблиці, графіки

✓ Імплікація $P \rightarrow Q$

Фраза 'якщо P, то Q' хибна тільки в одному випадку: P істинне, а Q хибне. Обіцянка порушена лише тоді, коли умову виконали, а результат не дали.

Якщо прибереш кімнату (P),
отримаєш цукерку (Q).
 $P=\text{true}, Q=\text{false} \rightarrow$ брехня.
Інші випадки $\rightarrow$ правило не порушено.

✓ Must be true

Потрібно вибрати те, що неминуче випливає з умови. Не те, що ймовірно, знайомо або звучить розумно.

Якщо $A > B$ і $B > C$,
то $A > C$ обов'язково.
А от 'A найбільше у світі' не випливає.

📊 Таблиці

Спершу читаємо назви рядків і колонок, потім одиниці виміру.
Пастка: порівняти відсотки з різних баз.

20% від 100 = 20
20% від 50 = 10
Однаковий відсоток не означає однакову кількість.

📈 Графіки функцій

Лінійна функція дає пряму.
Квадратична - параболу. Обернена пропорційність $y = 1/x$ має дві гілки.
Експонента швидко росте або спадає.

$y = x \rightarrow$ пряма
 $y = x^2 \rightarrow$ парабола
 $y = 1/x \rightarrow$ гіпербола
 $y = 2^x \rightarrow$ експонента

📊 Логарифм

Логарифм питає: до якого степеня треба піднести основу, щоб отримати число. Це обернена дія до експоненти.

$2^3 = 8$
 $\log_2(8) = 3$

📊 Відсотки

Завжди питай: від якої бази рахуємо? Збільшення і зменшення на той самий відсоток не повертає до початкового числа.

$100 + 20\% = 120$
 $120 - 20\% = 96$

Науково: логічні задачі перевіряють не пам'ять, а валідність переходу від умови до висновку. Комбінаторика перевіряє структуру простору варіантів: альтернативи додаються, незалежні кроки множаться, порядок або зберігається, або стирається діленням.

Приклад у житті: якщо доставка каже "якщо замовлення до 12:00, привеземо сьогодні", вона збрехала тільки тоді, коли замовлення було до 12:00, а доставка не приїхала сьогодні. Якщо замовлення було після 12:00, правило нічого не обіцяло.

ГВ English: що реально ловлять

Tenses

Час обираємо не за перекладом, а за маркером і логікою події. Present Perfect - результат до тепер. Past Simple - завершений час у минулому. Past Continuous - дія в процесі в минулому.

I have already finished.
I finished yesterday.
I was reading when he called.

Passive voice

Якщо важливий об'єкт дії, а не виконавець, часто потрібен passive: be + V3.

The report was prepared yesterday.
The data are stored in a database.

Conditionals

Умовні речення перевіряють реальність ситуації: реальна, малоймовірна, минула нереальна.

If it rains, we will stay home.
If I knew, I would tell you.
If I had known, I would have told you.

Modals

must - внутрішня необхідність або сильний висновок. have to - зовнішнє правило. should - порада. may/might - можливість.

You must be tired. -> сильний висновок
You have to submit the form. -> правило

Articles

a/an - один із багатьох, the - конкретний або вже відомий. Нульовий артикль часто з абстрактними/загальними поняттями.

a database -> якась база
the database -> конкретна база

Collocations

На тесті часто треба знати не переклад, а природну пару слів: make a decision, do research, depend on, responsible for.

make a mistake
do homework
depend on
interested in

Linkers

however - контраст, therefore - наслідок, although - контраст усередині речення, because - причина, despite - після нього noun/gerund.

Although it was late, we continued.
Despite being tired, we continued.

Reading

Не шукай 'красиву' відповідь.
Шукай рядок-доказ. Якщо відповідь ширша за текст або додає припущення, вона підозріла.

Question asks: why?
Find because / reason / caused by.
Then compare wording, not emotion.

- **Переклад прийменника дослівно**

Українське 'залежить від' не допомагає, англійською буде depend on.

- **Плутанина despite / although**

Despite + noun/gerund. Although + повне речення з підметом і присудком.

despite the rain / although it rained

- **Present Perfect vs Past Simple**

Already/just/yet часто тягнуть Perfect, yesterday/last year - Past Simple.

- **Reading: відповідь звучить логічно, але її нема в тексті**

На іспиті правильна відповідь має мати доказ, а не просто бути правдоподібною.



ІТ: піддисципліни, які треба впізнавати

Процедурне програмування

Програма організована як послідовність команд і процедур. Дані й функції часто існують окремо.

```
function volumeUp(device) {  
  device.volume += 1;  
}
```

ООП: клас і об'єкт

Клас - шаблон. Об'єкт - конкретний екземпляр. Інкапсуляція ховає внутрішній стан і дає керований доступ.

```
class Speaker {  
  volume = 10;  
  up() { this.volume++; }  
}  
const jbl = new Speaker();
```

Наслідування

Новий клас отримує властивості й методи базового класу. Це 'is-a': ноутбук є пристроєм. Пастка: не плутати з композицією, де об'єкт має інший об'єкт.

```
class Device {}  
class Laptop extends Device {}  
// Laptop is a Device
```

Поліморфізм

Одна команда викликає різну поведінку залежно від об'єкта. Як кнопка гучності: на телефоні змінює звук дзвінка, у плеєрі - медіа, у навушниках - локальний рівень.

```
devices.forEach(d =>  
  d.volumeUp());  
// method name same,  
// behavior can differ
```

Алгоритми і Big-O

Big-O описує, як росте час/пам'ять при збільшенні входу. $O(1)$ - стало, $O(\log n)$ - дуже повільний ріст, $O(n)$ - лінійно, $O(n^2)$ - вкладені порівняння.

```
binary search ->  $O(\log n)$   
linear scan ->  $O(n)$   
nested loops ->  $O(n^2)$ 
```

Структури даних

Array швидко бере за індексом. Stack - останній зайшов, перший вийшов. Queue - перший зайшов, перший вийшов. Hash table шукає за ключем.

```
stack.push(x); stack.pop();  
queue.enqueue(x); queue.dequeue();
```

SQL і БД

SELECT читає, JOIN з'єднує таблиці, WHERE фільтрує рядки до групування, HAVING фільтрує групи після GROUP BY, GRANT дає права.

```
SELECT user_id, COUNT(*)  
FROM orders  
WHERE status='paid'  
GROUP BY user_id  
HAVING COUNT(*) > 3;
```

Моделі даних

Концептуальна модель описує сутності бізнесу. Логічна - таблиці, ключі, зв'язки. Фізична - як це зберігає конкретна СУБД.

```
student - course - enrollment  
PK: student_id  
FK: enrollment.student_id
```

Мережі, sysadmin/devops, безпека, Data Science

OSI

OSI - модель із 7 рівнів. Для тесту важливо не вивчити все як вірш, а прив'язати приклади: фізика, кадр, пакет/IP, TCP/UDP, HTTP/DNS.

```
L2: Ethernet, MAC, switch
L3: IP, router
L4: TCP/UDP, ports
L7: HTTP, DNS, SMTP
```

Packet / frame / segment

На L2 часто говорять frame і MAC. На L3 - packet і IP. На L4 - segment/datagram і ports. Пастка: switch не маршрутизує за IP як router.

```
switch -> MAC table
router -> routing table / IP
DNS -> domain to IP
```

Sysadmin minimum

Користувачі, права, процеси, журнали, служби, резервні копії, оновлення. На тесті питають роль: хто керує ресурсами і доступом.

```
Linux:
ps aux
systemctl status nginx
chmod 640 file
journalctl -u service
```

DevOps minimum

DevOps - не тільки 'деплой'. Це CI/CD, автоматизація, контейнери, моніторинг, інфраструктура як код.

```
git push -> CI tests -> build ->
deploy
Dockerfile -> image
Kubernetes -> orchestration
```

Операційні системи

ОС керує процесами, пам'яттю, файловою системою, пристроями та доступом. Kernel - ядро, shell - інтерфейс команд.

```
process -> scheduler
RAM -> memory manager
file -> filesystem
USB -> driver
```

Криптографія

Hash - односторонній відбиток. Encryption - можна розшифрувати ключем. Digital signature - підтверджує автора й цілісність.

```
Kupyna -> hash
AES/Rijndael -> symmetric
encryption
RSA/ECDSA -> signatures /
asymmetric
```



ML / Data Science

Classification передбачає клас,
regression - число, clustering - групи
без міток. SVM шукає гіперплощину
з максимальним margin.

```
classification: spam / not spam
regression: price
clustering: customer groups
```



Data Engineering зв'язок

Для екзамену не треба йти вглиб
професії, але корисно бачити роль:
SQL, сховища, ETL/ELT, якість даних,
права доступу, моніторинг
пайплайнів.

```
source -> extract -> transform ->
warehouse
quality checks -> dashboard
```

- **Switch vs router**

Switch працює переважно на L2 і дивиться на MAC. Router працює на L3 і маршрутизує IP-пакети між мережами.

- **Hash vs encryption**

Hash не 'розшифровують'. Якщо можна повернути оригінал ключем - це шифрування, не хеш.

- **Inheritance vs composition**

Inheritance: Laptop is a Device. Composition: Laptop has a Battery.

- **WHERE vs HAVING**

WHERE до групування, HAVING після групування. Якщо є агрегат COUNT/SUM у фільтрі - часто HAVING.

- **Compiler / interpreter / linker**

Compiler перекладає код, interpreter виконує поступово, linker збирає об'єктні модулі в виконуваний модуль.